
.NET Advanced

Study Notes

1ITVTSOF1

Academic Year 2026–2027

Dries Van den Brande

Contents

Chapter 1 ASP.NET Core Basisconcepten	2
1.1 Web Frameworks	2
1.2 Evolutie	5
1.3 API project aanmaken	6
1.4 Controllers	6
1.4.1 HTTP request, response, middleware en routing	6
1.4.2 CRUD-actions, HTTP-verbs en routes	6
1.4.3 Parameters uit een request halen	7
1.4.4 ActionResult en HTTP-statuscodes	8
1.4.5 JSON serialisatie	8
1.4.6 API testen met een .http-bestand	9
1.4.7 Lifecycle van een Controller	9

ASP.NET Core Basisconcepten

Learning Objectives

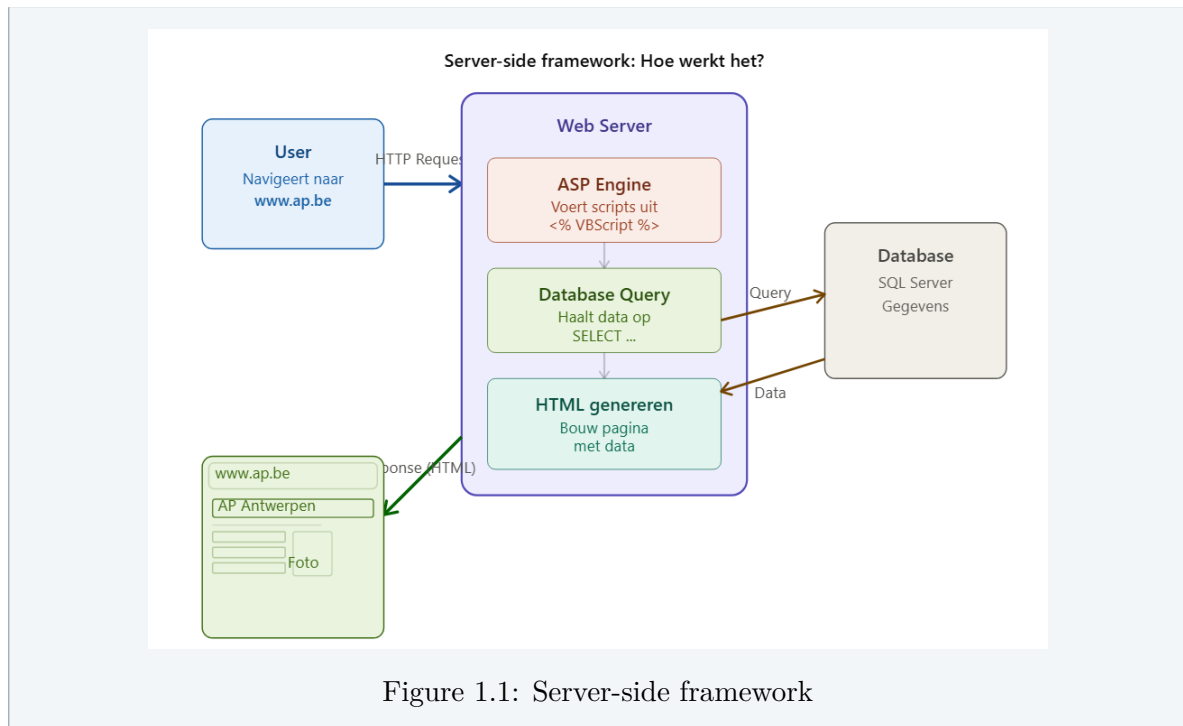
1. Je kan het verschil uitleggen tussen een server side en een client side web framework
2. Je kent de historische evolutie van ASP → ASP.NET → ASP.NET (Core) MVC en de .NET (Core)-versies tot vandaag
3. Je kan een nieuw ASP.NET Core Web API-project aanmaken en opstarten
4. Je begrijpt de structuur van een leeg Web API-project (Program.cs, appsettings.json, Controllers)
5. Je begrijpt hoe een HTTP request en response zijn opgebouwd en hoe een request via de middleware pipeline en de routing tot in je controller geraakt
6. Je kan een controller met CRUD-actions schrijven en de juiste HTTP-verbs en routes toekennen
7. Je kan parameters uit een request halen (path, query string, headers, body)
8. Je kan met ActionResult<T> en de helper-methodes zelf de juiste statuscode teruggeven en begrijpt hoe objecten automatisch naar JSON worden omgezet
9. Je kan je API testen met een .http-bestand in Visual Studio
10. Je begrijpt de levenscyclus van een Controller.

1.1 Web Frameworks

Het ASP Framework is een **server side web framework**

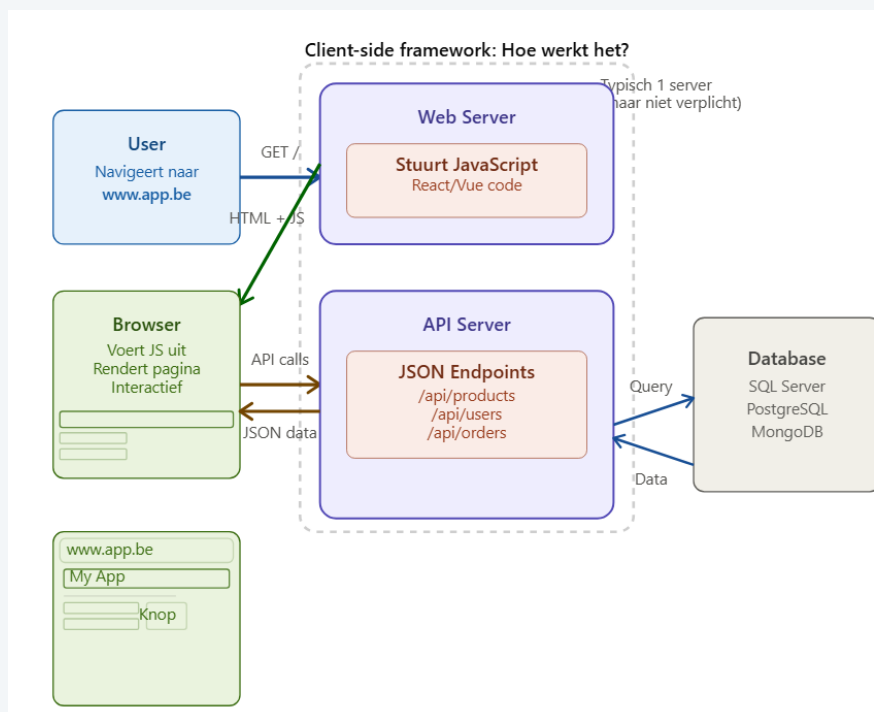
Server Side Web Framework

Een server-side framework is een framework waarbij de server de logica uitvoert en HTML genereert, waarna die HTML naar de browser wordt gestuurd om te tonen. Bij elke nieuwe pagina of actie stuurt de browser opnieuw een request naar de server.



Client Side Web Framework

Een client-side framework is een framework waarbij de UI-logica in de browser wordt uitgevoerd met JavaScript. De applicatie wordt in de browser geladen en vraagt daarna zelf gegevens op bij de server, meestal via een Web API die data zoals JSON terugstuurt.



Document Object Model (DOM)

De DOM is de objectgeoriënteerde representatie van een HTML-pagina in de browser. Elke HTML-tag wordt voorgesteld als een object in een boomstructuur, die door JavaScript kan worden aangepast en door de browser wordt gerenderd.

Domein

Een domein is het deel van een URL dat na `http://` of `https://` komt en de website identificeert, bijvoorbeeld `ap.be`.

Route

Een route is het deel van een URL dat na het domein komt en aangeeft welke resource of functionaliteit wordt opgevraagd. Bijvoorbeeld bij `https://ap.be/api/students` is `api/students` de route.

CORS

CORS staat voor *Cross-Origin Resource Sharing* en bepaalt of een applicatie resources mag opvragen van een ander domein dan het domein waarop de applicatie zelf gehost wordt.

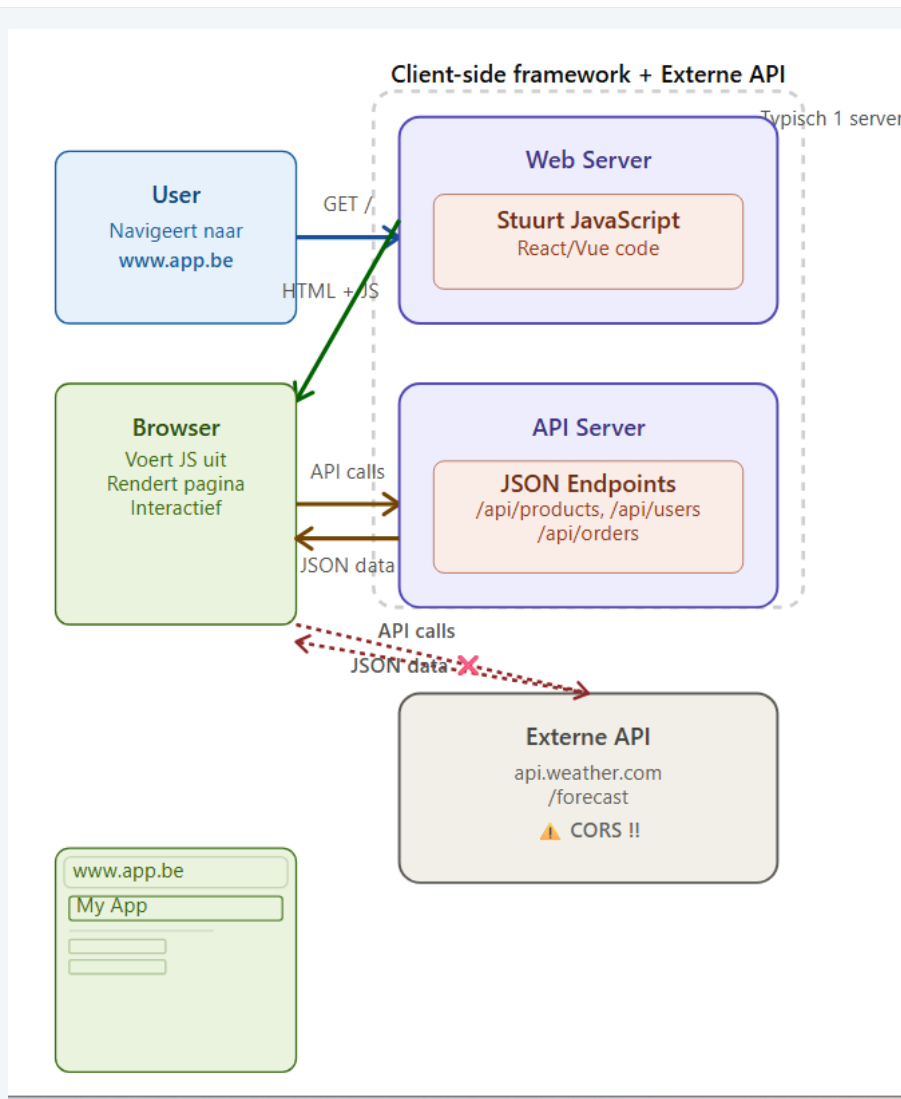


Figure 1.3: Client side framework met externe API en CORS

1.2 Evolutie

- **1996 – ASP (Active Server Pages):** HTML gecombineerd met server-side scripts om dynamische webpagina's te genereren.
- **2002 – ASP.NET:** server-side code kon geschreven worden in .NET-talen zoals C# en VB.NET.
- **2009 – ASP.NET MVC:** introduceerde het Model-View-Controller patroon om de applicatie op te splitsen in data (Model), gebruikersinterface (View) en logica (Controller).
- **2016 – ASP.NET Core MVC:** volledig herbouwde, open-source en cross-platform versie van ASP.NET met betere prestaties en een modulaire architectuur.
- **Vanaf 2017:** ASP.NET Core evolueert samen met de verschillende .NET Core- en later .NET-versies.

LTS

LTS staat voor *Long Term Support*. Een LTS-versie wordt gedurende een langere periode ondersteund met beveiligingsupdates, bugfixes en stabiliteitsverbeteringen, waardoor ze geschikt is voor projecten die langdurig stabiel moeten blijven.

1.3 API project aanmaken

- **Program.cs:** het startpunt van de applicatie. Hier wordt de webserver gestart en worden services, dependency injection, middleware en routes geconfigureerd.
- **appsettings.json:** bevat configuratiegegevens van de applicatie, zoals logging en databank-connectiestrings.
- **Controllers/:** bevat de controllers die binnenkomende HTTP-requests verwerken en bepalen welke response wordt teruggestuurd.

1.4 Controllers

1.4.1 HTTP request, response, middleware en routing

Een **HTTP request** bestaat uit drie delen:

- **Request line:** bevat de HTTP-verb en het path.
- **Headers:** extra informatie over de request.
- **Body:** optionele data die naar de server wordt gestuurd.

Bijvoorbeeld:

```
PUT /api/products/5 HTTP/1.1
Content-Type: application/json

{
  "name": "Umbrella",
  "price": 15.99
}
```

Een **HTTP response** bestaat eveneens uit drie delen:

- **Status line:** bevat de HTTP-statuscode.
- **Headers:** extra informatie over de response.
- **Body:** optionele gegevens die worden teruggestuurd.

Een binnenkomende request doorloopt eerst de **middleware pipeline**. Elke middleware voert een specifieke taak uit en stuurt de request normaal door naar de volgende middleware. Een middleware kan de request ook stoppen en zelf onmiddellijk een response terugsturen.

De **routing middleware** bepaalt uiteindelijk welke controller en action moeten worden uitgevoerd op basis van de route en HTTP-verb.

1.4.2 CRUD-actions, HTTP-verbs en routes

CRUD staat voor:

CRUD	HTTP-verb	Attribuut
Create	POST	[HttpPost]
Read	GET	[HttpGet]
Update	PUT / PATCH	[HttpPut] / [HttpPatch]
Delete	DELETE	[HttpDelete]

Een controller krijgt typisch een basisroute:

```
[Route("api/[controller]")]
[ApiController]
public class BooksController : ControllerBase
{
}
```

Voor een specifieke resource kan een routeparameter worden toegevoegd:

```
[HttpGet("{id}")]
public ActionResult<Book> GetBook(int id)
{
    ...
}
```

Hierbij wordt de volledige route bijvoorbeeld:

/api/books/5

Twee actions mogen dezelfde HTTP-verb gebruiken zolang hun routes verschillend zijn.

1.4.3 Parameters uit een request halen

Een client kan extra informatie op verschillende manieren meesturen:

- **Path / Route:** onderdeel van de URL
- **Query string:** parameters achter ?
- **Headers:** key/value-informatie
- **Body:** meestal een JSON-object

ASP.NET Core voorziet hiervoor attributen:

- **[FromRoute]:** haalt een parameter uit het path.
- **[FromQuery]:** haalt een parameter uit de query string.
- **[FromHeader]:** haalt een waarde uit de headers.
- **[FromBody]:** haalt JSON uit de body en zet deze om naar een object.

Voorbeeld:

```
[HttpGet]
public ActionResult<IEnumerable<Person>> GetStudents(
    [FromHeader] string? licenseKey,
    [FromQuery] string? name,
    [FromQuery] int? age)
{
    ...
}

[HttpPost]
public ActionResult<Person> CreateStudent(
    [FromBody] Person student)
{
    ...
}
```

1.4.4 ActionResult en HTTP-statuscodes

Een controller-action kan als returntype `ActionResult<T>` gebruiken. Hierdoor kan de action zowel data als een specifieke HTTP-statuscode teruggeven.

Veelgebruikte helper-methodes:

Helper	Statuscode
<code>Ok(...)</code>	200 OK
<code>Created(...)</code>	201 Created
<code>NoContent()</code>	204 No Content
<code>BadRequest()</code>	400 Bad Request
<code>NotFound()</code>	404 Not Found

Typische CRUD-statuscodes:

- GET: 200 OK
- POST: 201 Created
- PUT: 200 OK
- DELETE: 204 No Content

Voorbeeld:

```
[HttpGet("{id}")]
public ActionResult<Book> GetBook(int id)
{
    Book? book = ...;

    if (book == null)
        return NotFound();

    return Ok(book);
}
```

1.4.5 JSON serialisatie

ASP.NET Core zet C#-objecten die vanuit een controller worden teruggegeven automatisch om naar JSON.

Dit noemt men **serialisatie**.

De omgekeerde bewerking, waarbij JSON wordt omgezet naar een C#-object, noemt men **deserialisatie**.

Bijvoorbeeld:

```
return Ok(book);
```

kan in de HTTP-response automatisch worden:

```
{
  "id": 1,
  "title": "The Hobbit"
}
```

ASP.NET Core zet daarbij automatisch:

```
Content-Type: application/json
```

1.4.6 API testen met een .http-bestand

Een browser kan normaal enkel eenvoudig GET-requests uitvoeren.

Om ook POST-, PUT- en DELETE-requests te testen kan je in Visual Studio een `.http`-bestand gebruiken.

Voorbeeld:

```
GET https://localhost:7000/api/books
```

```
###
```

```
GET https://localhost:7000/api/books/5
```

```
###
```

```
POST https://localhost:7000/api/books
```

```
Content-Type: application/json
```

```
{  
    "title": "The Hobbit",  
    "numberOfPages": 310  
}
```

```
###
```

```
DELETE https://localhost:7000/api/books/5
```

1.4.7 Lifecycle van een Controller

Je maakt een controller-object **niet zelf aan**.

Wanneer een request overeenkomt met een bepaalde controller en action, maakt ASP.NET Core automatisch een nieuwe instantie van die controller aan.

Die controller bestaat enkel tijdens de verwerking van die ene request en response.

Daarna wordt de instantie opnieuw verwijderd.

Hierdoor mag je geen permanente gegevens bewaren in gewone velden van een controller.